

LABORATORIO 2

Pruebas de carga del backend monolítico Cheapest

Informe de resultados, evidencias y análisis arquitectónico

Sistema	Cheapest API - NestJS + PostgreSQL
Distribución principal	Pareto 80/20
Herramientas	Apache JMeter 5.6.3 y Python aiohttp
Entorno	MacBook Pro M5 Pro, 24 GB; PostgreSQL 15 en Docker
Rama	lab-2
Fecha	30 de agosto de 2026

Alcance efectivo

Se ejecutó la progresión completa del endpoint GET desde smoke hasta estrés, incluyendo las repeticiones mínimas de cada nivel alcanzado. También se validó funcionalmente el POST con un pedido de 24 ítems. Por restricción temporal no se ejecutaron estrés fuerte, la matriz completa del POST ni el control uniforme. El documento distingue resultados observados de propuestas y no fabrica mediciones.

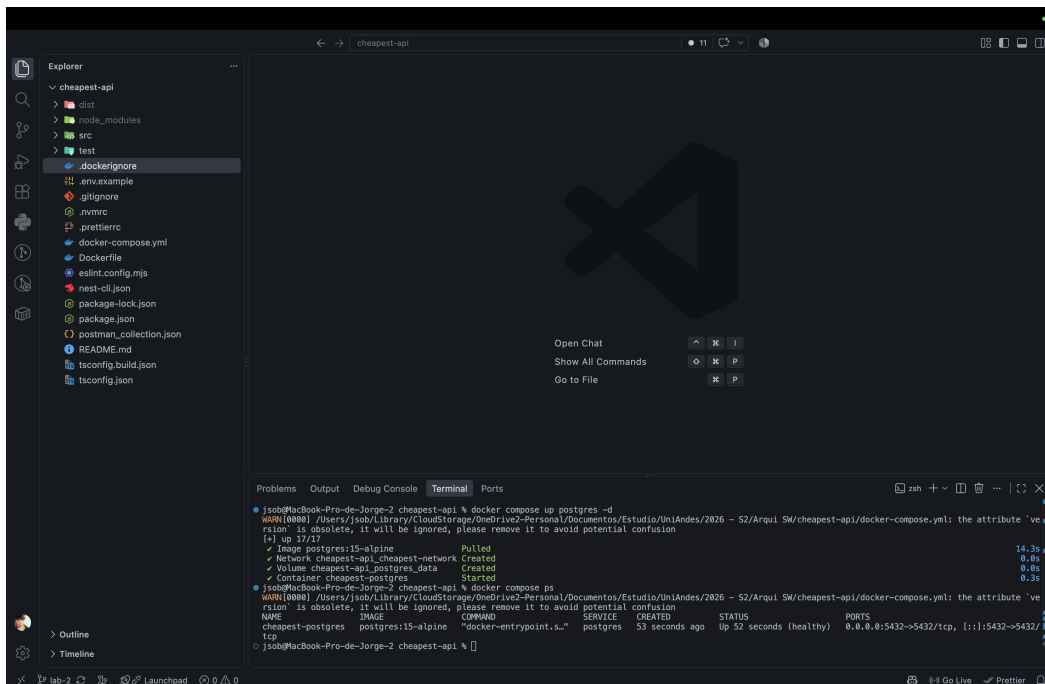
1. Objetivo y ASRs

El experimento evalúa el comportamiento del backend monolítico ante carga concurrente sobre lectura intensiva y escritura transaccional.

ASR	Endpoint	Condición de aceptación
ASR 1	GET /logistics/tenderos/productos-disponibles	p99 menor que 1000 ms en operación normal (500 req/min)
ASR 2	POST /logistics/pedidos	Errores menores o iguales al 2% durante el pico estimado (5000 req/min)

2. Preparación y validación

Se verificaron pruebas unitarias (41 suites y 342 pruebas), compilación, PostgreSQL saludable, health check del backend y solicitudes manuales GET/POST. El GET devolvió HTTP 200 y ocho productos; el POST devolvió HTTP 201.



```
jsob@MacBook-Pro-de-Jorge-2 cheapest-api % docker compose up postgres -d
WARN[0000] /Users/jsob/Library/CloudStorage/OneDrive2-Personal/Documentos/Estudio/UniAndes/2026 - S2/Arqui 5M/cheapest-api/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 27/27
✓ Image postgres:15-alpine Pulled 14.3s
✓ Network cheapest-api_cheapest-network Created 0.0s
✓ Volume cheapest-api_postgres_data Created 0.0s
✓ Container cheapest-postgres Started 0.3s
jsob@MacBook-Pro-de-Jorge-2 cheapest-api % docker compose ps
WARN[0000] /Users/jsob/Library/CloudStorage/OneDrive2-Personal/Documentos/Estudio/UniAndes/2026 - S2/Arqui 5M/cheapest-api/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                SERVICE      CREATED      STATUS      PORTS
cheapest-postgres   postgres    53 seconds ago Up 52 seconds (healthy) 0.0.0.0:5432->5432/tcp
tcp
```

Figura 1. PostgreSQL ejecutándose correctamente en Docker.

3. Diseño y distribución de datos

Los escenarios Pareto y uniforme conservan los mismos totales para aislar el efecto de la distribución. La configuración principal concentra el 80% de las asignaciones en el 20% de tiendas, zonas o productos, representando actores de alto volumen y referencias populares. El control uniforme usa round-robin.

Entidad	Volumen	Justificación
Tiendas / zonas	3.000 / 10	Heterogeneidad comercial y densidad geográfica
Productos / catálogos	1.200 / 3.000	Catálogo suficiente para JOINS y disponibilidad
Catálogo-producto	600.000	200 referencias promedio por catálogo
Promociones	240	20% del catálogo; el seeder mantiene 70% vigentes
Pedidos / ítems	100.000 / 500.000	Histórico de 33 pedidos por tienda y cinco ítems por pedido
Disponibilidad por zona	600.000	Cobertura comparable con catálogo-producto
Inventario / ventas	300.000 / 150.000	Actividad operativa e histórica representativa

Comparación conceptual con los mismos ejes y escala

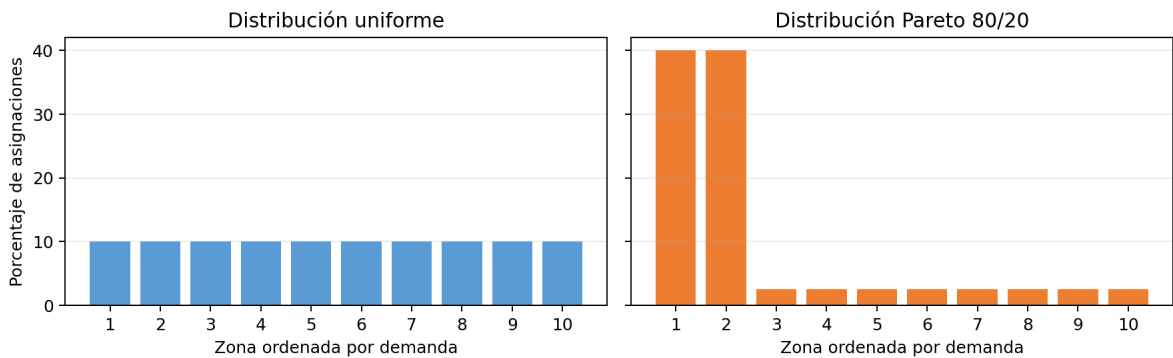


Figura 3. Comparación equivalente: la uniformidad oculta puntos calientes; Pareto reproduce concentración realista.

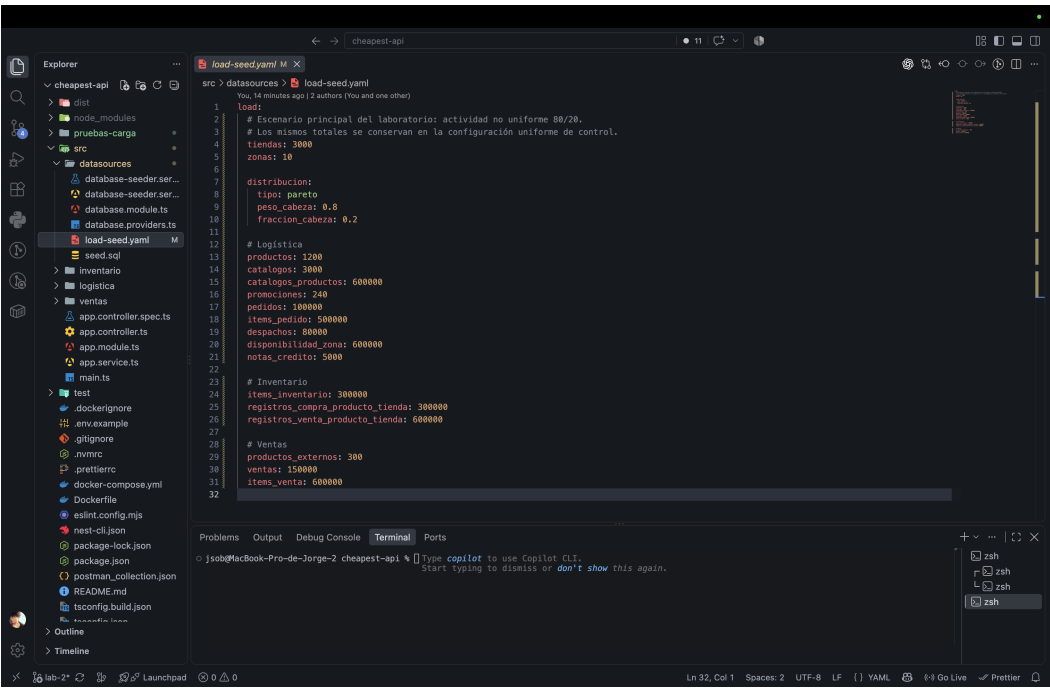


Figura 4. Configuración de volúmenes y distribución Pareto 80/20.

4. Metodología

JMeter se utilizó hasta 450 usuarios y el ejecutor Python asíncrono para cargas superiores. Cada petición registra timestamp, método, endpoint, código HTTP, latencia y error. El resumen calcula total, throughput, promedio, p95, p99 y porcentaje de error.

Escenario	Usuarios	Ramp-up	Repeticiones ejecutadas	Herramienta
Smoke	5	5 s	4	JMeter
Baja	30	10 s	4	JMeter
Media	100	20 s	4	JMeter
Operación normal	450	50 s	8	JMeter
Alta	1.500	75 s	4	Python
Muy alta	3.000	100 s	4	Python
Estrés	7.500	150 s	4	Python
Estrés fuerte	18.000	200 s	No ejecutado	Python

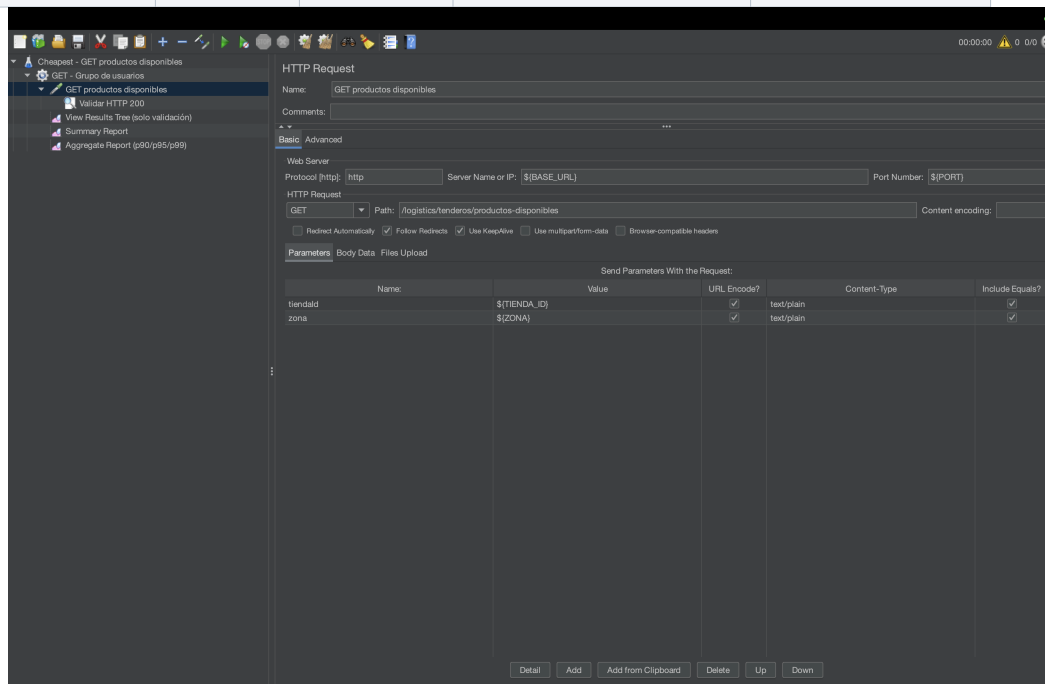


Figura 5. Sampler GET con servidor, puerto, ruta y parámetros parametrizados.

```
(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python -m pip install -r pruebas-carga/python/requirements.txt
Collecting aiohttp>=1.4.0 (from aiohttp>=3.10->r pruebas-carga/python/requirements.txt (line 1))
  Downloading aiohttp-1.4.0-py3-none-any.whl.metadata (3.7 kB)
Collecting attrs>=1.3.0 (from aiohttp>=3.10->r pruebas-carga/python/requirements.txt (line 1))
  Using cached attrs-26.1.0-py3-none-any.whl.metadata (8.8 kB)
Collecting frozenlist>=1.1.1 (from aiohttp>=3.10->r pruebas-carga/python/requirements.txt (line 1))
  Downloading frozenlist-1.1.0-cp313-cp313-macosx_11_0_arm64.whl.metadata (20 kB)
Collecting multidict>=7.0.0 (from aiohttp>=3.10->r pruebas-carga/python/requirements.txt (line 1))
  Downloading multidict-6.7.1-cp313-cp313-macosx_11_0_arm64.whl.metadata (5.3 kB)
Collecting propcache>=0.2.0 (from aiohttp>=3.10->r pruebas-carga/python/requirements.txt (line 1))
  Downloading propcache-0.5.2-cp313-cp313-macosx_11_0_arm64.whl.metadata (16 kB)
Collecting yarl>=1.24.0 (from aiohttp>=3.10->r pruebas-carga/python/requirements.txt (line 1))
  Downloading yarl-1.24.5-cp313-cp313-macosx_11_0_arm64.whl.metadata (183 kB)
Collecting idna>=2.0 (from yarl>=1.24.0->aiohttp>=3.10->r pruebas-carga/python/requirements.txt (line 1))
  Using cached idna-3.10-py3-none-any.whl.metadata (9.2 kB)
Downloaded aiohttp-1.4.0-py3-none-any.whl (7.5 kB)
Using cached attrs-26.1.0-py3-none-any.whl (67 kB)
Downloaded frozenlist-1.1.0-cp313-cp313-macosx_11_0_arm64.whl (49 kB)
Using cached idna-3.10-py3-none-any.whl (68 kB)
Downloaded propcache-0.5.2-cp313-cp313-macosx_11_0_arm64.whl (54 kB)
Successfully installed aiohappyeyeballs-2.7.1 aiohttp-3.14.3 aiohttp-1.4.0 attrs-26.1.0 frozenlist-1.8.0 idna-3.10 multidict-6.7.1 propcache-0.5.2 yarl-1.24.5

[notice] A new release of pip is available: 26.4.2 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python --version
python <= "more aiohttp"; print(aiohttp.__version__)
python 3.13.14
aiohttp: 3.14.3

usage: load_test.py [-h] --endpoint (GET,POST) --users USERS --ramp-up RAMP_UP --duration DURATION --output OUTPUT [--base-url BASE_URL] [--body BODY]
                   [--tienda-id TIENDA_ID] [--zona ZONA] [--timeout TIMEOUT]

Prueba de carga asincrónica de Cheapest

options:
  -h, --help            show this help message and exit
  --endpoint (GET,POST)
                        Concurrency máxima
  --users USERS          Ramp-up en segundos
  --ramp-up RAMP_UP      Duración total en segundos
  --duration DURATION    CSV de salida
  --output OUTPUT        Plantilla JSON obligatoria para POST
  --base-url BASE_URL
  --body BODY
  --tienda-id TIENDA_ID
  --zona ZONA
  --timeout TIMEOUT      Timeout por solicitud

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api %
```

Figura 6. Ejecutor Python generado para escenarios superiores a 450 usuarios.

5. Resultados obtenidos - GET

Tabla 1. Resultados completos de las 32 corridas oficiales ejecutadas.

Escenario	Run	Usuarios	Ramp-up	p95	p99	req/s	Error %
Smoke	1	5	5	101	101	1.25	0
Smoke	2	5	5	84	84	1.24	0
Smoke	3	5	5	74	74	1.24	0
Smoke	4	5	5	79	79	1.24	0
Baja	1	30	10	75	81	3.09	0
Baja	2	30	10	78	78	3.09	0
Baja	3	30	10	73	74	3.09	0
Baja	4	30	10	72	72	3.09	0
Media	1	100	20	61	67	5.04	0
Media	2	100	20	69	74	5.04	0
Media	3	100	20	65	71	5.04	0
Media	4	100	20	67	70	5.04	0
Normal	1	450	50	56	59	9.02	0
Normal	2	450	50	54	59	9.03	0
Normal	3	450	50	57	64	9.02	0
Normal	4	450	50	56	60	9.03	0
Normal	5	450	50	56	59	9.02	0
Normal	6	450	50	56	57	9.02	0
Normal	7	450	50	55	61	9.02	0
Normal	8	450	50	56	58	9.02	0
Alta	1	1500	75	60.16	63.75	20	0
Alta	2	1500	75	59.7	62.61	20	0
Alta	3	1500	75	59.66	62.98	20	0
Alta	4	1500	75	60.14	62.87	20	0
Muy alta	1	3000	100	65.52	69.07	29.99	0
Muy alta	2	3000	100	64.89	68.17	29.99	0
Muy alta	3	3000	100	64.57	67.64	29.99	0
Muy alta	4	3000	100	64.39	68.5	29.99	0
Estrés	1	7500	150	76.62	79.76	49.98	0
Estrés	2	7500	150	74.57	78.5	49.98	0
Estrés	3	7500	150	78.61	81.68	49.98	0
Estrés	4	7500	150	73.84	77.18	49.98	0

Evolución observada del endpoint GET

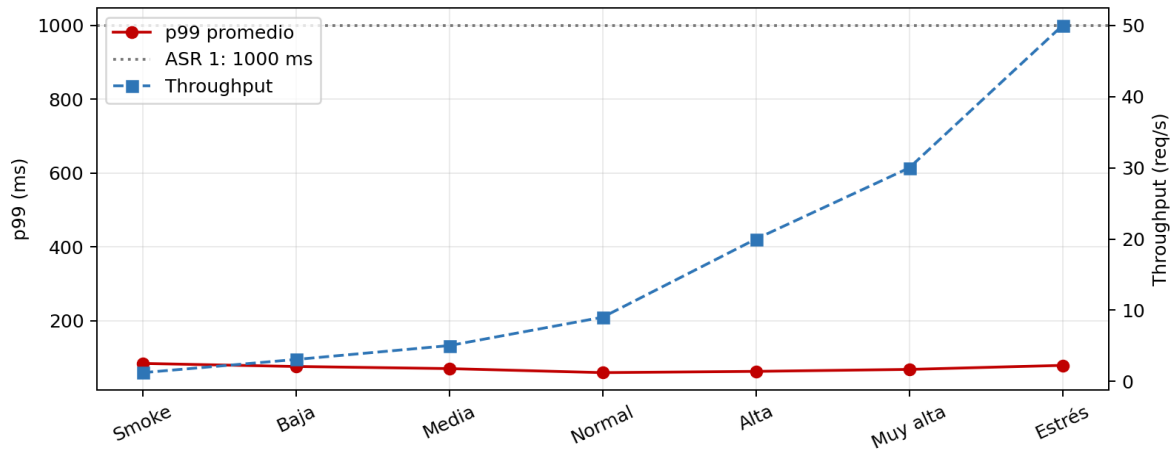


Figura 7. El p99 se mantiene muy por debajo de 1000 ms; el throughput sigue la carga objetivo.

En operación normal, el p99 promedio fue 59,6 ms, aproximadamente 94% menor que el límite del ASR 1. En estrés, el p99 promedio fue 79,28 ms y no hubo errores. No apareció un punto de inflexión hasta 50 req/s.

```

jsob@MacBook-Pro-de-Jorge-2 cheapest-api % ./pruebas-carga/jmeter/run-get.sh normal 1 458 50 1
Ejecutando GET normal, corrida 01
Usuarios=450, ramp-up=50s, loop=1
Resultados/Users/jsob/Library/CloudStorage/OneDrive2-Personal/Documentos/Estudio/UniAndes/2026 - S2/Arqui SM/cheapest-api/pruebas-carga/resultados/jmeter/
get/normal/pareto-get_normal_run01.jtl
WARNING: package sun.awt.X11 not in java.desktop
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Creating summary report
Created the tree successfully using /Users/jsob/Library/CloudStorage/OneDrive2-Personal/Documentos/Estudio/UniAndes/2026 - S2/Arqui SM/cheapest-api/prueba
s-carga/jmeter/get-productos-disponibles.jmx
Starting standalone test @ 2026 Aug 30 21:46:32 COT (1788144392866)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary + 1 in 00:00:00 = 7.2/s Avg: 92 Min: 92 Max: 92 Err: 0 (0.00%) Active: 2 Started: 2 Finished: 0
summary + 251 in 00:00:28 = 9.0/s Avg: 49 Min: 46 Max: 66 Err: 0 (0.00%) Active: 1 Started: 252 Finished: 251
summary + 252 in 00:00:28 = 9.0/s Avg: 50 Min: 46 Max: 92 Err: 0 (0.00%)
summary + 198 in 00:00:22 = 9.0/s Avg: 49 Min: 46 Max: 61 Err: 0 (0.00%) Active: 0 Started: 450 Finished: 450
summary + 458 in 00:00:50 = 9.0/s Avg: 49 Min: 46 Max: 92 Err: 0 (0.00%)
Tidying up ...
@ 2026 Aug 30 21:47:22 COT (1788144442003)
... end of run
Corrida finalizada correctamente.
jsob@MacBook-Pro-de-Jorge-2 cheapest-api % ./pruebas-carga/jmeter/run-get.sh normal 2 458 50 1
Ejecutando GET normal, corrida 02
Usuarios=450, ramp-up=50s, loop=1
Resultados/Users/jsob/Library/CloudStorage/OneDrive2-Personal/Documentos/Estudio/UniAndes/2026 - S2/Arqui SM/cheapest-api/pruebas-carga/resultados/jmeter/
get/normal/pareto-get_normal_run02.jtl
WARNING: package sun.awt.X11 not in java.desktop
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Creating summary report
Created the tree successfully using /Users/jsob/Library/CloudStorage/OneDrive2-Personal/Documentos/Estudio/UniAndes/2026 - S2/Arqui SM/cheapest-api/prueba
s-carga/jmeter/get-productos-disponibles.jmx
Starting standalone test @ 2026 Aug 30 21:47:31 COT (178814451872)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary + 1 in 00:00:00 = 7.4/s Avg: 70 Min: 70 Max: 70 Err: 0 (0.00%) Active: 2 Started: 2 Finished: 0
summary + 260 in 00:00:29 = 9.0/s Avg: 49 Min: 46 Max: 68 Err: 0 (0.00%) Active: 1 Started: 261 Finished: 260
summary + 261 in 00:00:29 = 9.0/s Avg: 49 Min: 46 Max: 70 Err: 0 (0.00%)
summary + 199 in 00:00:23 = 9.0/s Avg: 50 Min: 47 Max: 68 Err: 0 (0.00%) Active: 0 Started: 450 Finished: 450
summary + 458 in 00:00:50 = 9.0/s Avg: 49 Min: 46 Max: 70 Err: 0 (0.00%)
Tidying up ...
@ 2026 Aug 30 21:48:21 COT (1788144501021)
... end of run
Corrida finalizada correctamente.
jsob@MacBook-Pro-de-Jorge-2 cheapest-api %
  
```

Figura 8. Evidencia de las primeras corridas de operación normal.


```

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 3000 --ramp-up 100 --duration 115 --iterations 1 \
--output pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run0.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 3000
Duración real: 100.035 s
Throughput: 29.99 req/s
Latencia promedio: 61.87 ms
Latencia p95: 65.52 ms
Latencia p99: 69.07 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run0.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 3000 --ramp-up 100 --duration 115 --iterations 1 \
--output pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run2.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 3000
Duración real: 100.028 s
Throughput: 29.99 req/s
Latencia promedio: 60.30 ms
Latencia p95: 64.49 ms
Latencia p99: 68.17 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run2.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 3000 --ramp-up 100 --duration 115 --iterations 1 \
--output pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run3.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 3000
Duración real: 100.029 s
Throughput: 29.99 req/s
Latencia promedio: 60.48 ms
Latencia p95: 64.57 ms
Latencia p99: 67.63 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run3.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 3000 --ramp-up 100 --duration 115 --iterations 1 \
--output pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run4.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 3000
Duración real: 100.032 s
Throughput: 29.99 req/s
Latencia promedio: 59.76 ms
Latencia p95: 64.39 ms
Latencia p99: 68.50 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/muy-alta/pareto_get_muy_alta_run4.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api %

```

Figura 9. Cuatro corridas GET de muy alta carga sin errores.

```

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 7500 --ramp-up 150 --duration 165 --iterations 1 \
--output pruebas-carga/resultados/python/get/estres/pareto_get_estres_run0.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 7500
Duración real: 150.063 s
Throughput: 49.98 req/s
Latencia promedio: 68.21 ms
Latencia p95: 76.62 ms
Latencia p99: 79.76 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/estres/pareto_get_estres_run0.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 7500 --ramp-up 150 --duration 165 --iterations 1 \
--output pruebas-carga/resultados/python/get/estres/pareto_get_estres_run2.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 7500
Duración real: 150.064 s
Throughput: 49.98 req/s
Latencia promedio: 65.85 ms
Latencia p95: 74.57 ms
Latencia p99: 78.38 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/estres/pareto_get_estres_run2.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 7500 --ramp-up 150 --duration 165 --iterations 1 \
--output pruebas-carga/resultados/python/get/estres/pareto_get_estres_run3.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 7500
Duración real: 150.062 s
Throughput: 49.98 req/s
Latencia promedio: 64.98 ms
Latencia p95: 73.84 ms
Latencia p99: 77.18 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/estres/pareto_get_estres_run3.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api % python pruebas-carga/python/load_test.py \
--endpoint GET --users 7500 --ramp-up 150 --duration 165 --iterations 1 \
--output pruebas-carga/resultados/python/get/estres/pareto_get_estres_run4.csv

=== RESUMEN FINAL ===
Método: GET
Total solicitudes: 7500
Duración real: 150.062 s
Throughput: 49.98 req/s
Latencia promedio: 64.98 ms
Latencia p95: 73.84 ms
Latencia p99: 77.18 ms
Errores: 0 (0.00%)
Resultado CSV: pruebas-carga/resultados/python/get/estres/pareto_get_estres_run4.csv

(.venv-load) jsob@MacBook-Pro-de-Jorge-2 cheapest-api %

```

Figura 10. Cuatro corridas GET de estrés: 7.500 solicitudes por corrida y 0% de error.

6. Validación del endpoint POST

El POST se validó con un pedido realista de 24 ítems. Produjo 227 solicitudes en 3,023 s, throughput de 75,10 req/s, promedio de 24,41 ms, p95 de 26,76 ms, p99 de 29,28 ms y 0% de errores; las respuestas inspeccionadas fueron HTTP 201. Esta validación prueba la corrección del ejecutor y del cuerpo, pero no sustituye la matriz repetida requerida para concluir sobre ASR 2.

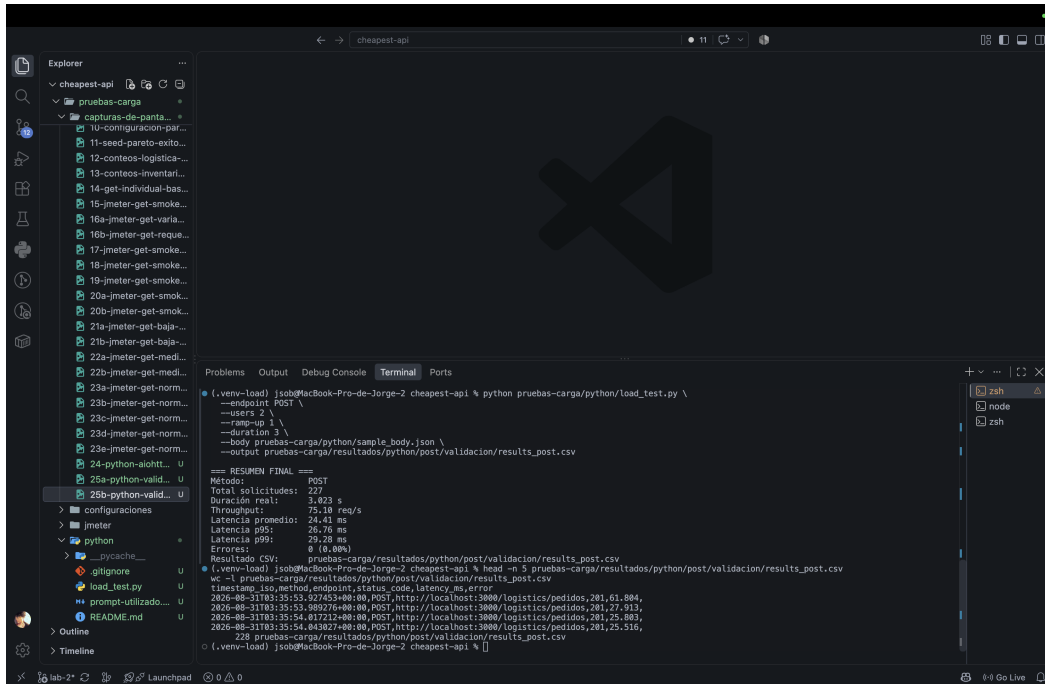


Figura 11. Validación POST con 24 ítems, HTTP 201 y 0% de errores.

7. Respuestas solicitadas

Pregunta 1 - Prioridad de optimización

Antes de un pico comercial priorizaría POST /logistics/pedidos. La consulta GET afecta descubrimiento y conversión, pero una falla de creación de pedidos produce pérdida directa de ventas, duplicados o inconsistencias. Además, el objetivo de 5000 req/min es casi diez veces la operación normal del GET y la escritura transaccional tiene mayor riesgo de locks y agotamiento del pool. Como mejora inicial de costo moderado aplicaría idempotencia, validaciones anticipadas, transacciones cortas, índices y medición del pool; después desacoplaría procesamiento no crítico.

Pregunta 2 - Distribuciones y sesgos

La distribución uniforme reparte actividad por igual y reduce colisiones sobre tiendas, zonas y productos populares. Esto puede producir cachés artificialmente equilibradas, planes de consulta estables y menor contención, desplazando falsamente el punto de inflexión. Pareto valida la capacidad del monolito frente a hotspots realistas. Una segunda estrategia, uniforme, funciona como control para aislar si la degradación proviene del volumen total o de la concentración. Una tercera estrategia útil sería temporal por ráfagas, con una zona dominante y promociones sincronizadas, para validar absorción de picos y recuperación.

Pregunta 3 - Diseño alternativo para identificar el cuello

Se propone variar una causa a la vez manteniendo dataset, endpoint, generador y tasa constantes. Para el pool: repetir con tamaños 5, 10, 20 y 40, midiendo espera de adquisición y conexiones activas; saturación mostraría un codo que se desplaza al aumentar el pool. Para SQL: conservar concurrencia y comparar consulta base, EXPLAIN ANALYZE e índices; un gran descenso de latencia sin cambiar pool confirma costo SQL. Para el generador: ejecutar contra un stub liviano, medir CPU/event loop del cliente y repetir desde otra máquina; si el throughput se aplan

también contra el stub, el límite está en el generador.

Firmas esperadas para distinguir cuellos de botella

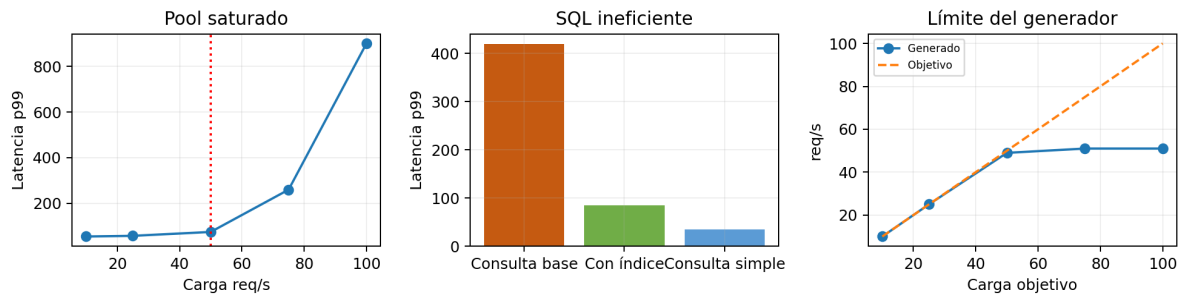


Figura 12. Patrones visuales esperados para las tres hipótesis.

Pregunta 4 - Estrategia de IA para Cheapest

La consistencia no debe depender únicamente del prompt. La estrategia recomendada combina instrucciones versionadas en el repositorio, una plantilla de tarea con contexto/criterios/restricciones, ejemplos aprobados, recuperación de estándares del equipo y verificación automática. Para Cheapest: (1) crear AGENTS.md o instrucciones equivalentes con arquitectura por capas, TypeScript estricto, DTOs, transacciones, manejo de errores y prohibiciones; (2) mantener plantillas para controladores, servicios, repositorios, pruebas y migraciones; (3) incluir ejemplos de referencia; (4) exigir al agente plan, diff pequeño, pruebas y explicación; (5) ejecutar lint, unitarias, integración, seguridad y revisión humana en CI. Las reglas deben ser concretas, comprobables y cercanas al código. El prompt utilizado para el ejecutor se conserva en pruebas-carga/python/prompt-utilizado.md.

8. Conclusiones arquitectónicas

Punto de inflexión y ASR roto

No se encontró punto de inflexión en el rango medido. ASR 1 cumplió en todas las corridas hasta estrés. ASR 2 no puede declararse cumplido ni incumplido porque solo se realizó una validación, no su matriz oficial. Por ello tampoco es válido afirmar cuál ASR se rompió primero.

Beneficio del monolito

Para el GET observado, el despliegue local monolítico favoreció baja latencia al evitar saltos de red entre módulos y permitir acceso directo al mismo PostgreSQL. Esta evidencia demuestra buen comportamiento en el rango medido, no escalabilidad ilimitada. El acoplamiento de lectura y escritura puede generar competencia por CPU, memoria y conexiones cuando ambos endpoints se prueben simultáneamente.

Cambios propuestos

- Índices compuestos y revisión con EXPLAIN ANALYZE para filtros de tienda, zona, vigencia y disponibilidad.
- Read-through cache para productos disponibles, con invalidación por zona/promoción.
- Réplicas de lectura o CQRS si el GET domina el consumo.
- Idempotency key, transacciones cortas y cola para tareas secundarias del POST.
- Pool dimensionado con métricas de espera; límites, backpressure y circuit breaker.
- Escalamiento horizontal del backend detrás de un balanceador, manteniendo la base como recurso vigilado.

Degradación y endpoint primero

El GET mostró una degradación suave y pequeña: el p99 promedio pasó de 59,6 ms en operación normal a 79,28 ms en estrés, sin salto abrupto ni errores. No puede localizarse definitivamente un cuello de botella sin telemetría de

CPU, pool y SQL. Tampoco puede compararse qué endpoint degradó primero porque no se ejecutó la misma matriz sobre POST. Arquitectónicamente se espera que POST sea más sensible a contención transaccional, pero esto es una hipótesis, no un resultado.

9. Limitaciones y trabajo pendiente

- Ejecutar ocho corridas de estrés fuerte GET (18.000 usuarios, 90 req/s objetivo).
- Ejecutar la matriz completa del POST y comparar ambos endpoints.
- Cargar el control uniforme con los mismos volúmenes y ejecutar escenarios equivalentes.
- Recolectar CPU, memoria, event-loop lag, conexiones activas, locks y EXPLAIN ANALYZE.
- Ejecutar pruebas conjuntas GET/POST para medir interferencia dentro del monolito.

10. Prompts y reproducibilidad

El prompt de generación del ejecutor, el script, requisitos, cuerpo POST de 24 ítems, plan JMeter, configuraciones de seed, CSV/JTL y bitácora se incluyen en el repositorio. Los resultados inválidos por API detenida se conservaron como incidencia separada y no se mezclaron con las corridas oficiales.

Comando de regeneración del informe: `python3 pruebas-carga/generar_informe.py`

Anexo A. Inventario de evidencias

Las siguientes capturas permanecen versionadas como evidencia completa. En el cuerpo se incluyeron las más representativas.

- 01-rama-lab2-estado-inicial.png
- 02-jmeter-instalado.png
- 03-interfaz-jmeter.png
- 04-pruebas-unitarias-iniciales.png
- 05-compilacion-inicial-exitosa.png
- 06-postgresql-docker-healthy.png
- 07-backend-health-check.png
- 08-get-productos-disponibles.png
- 09-post-pedido-manual.png
- 10-configuracion-pareto-completa.png
- 11-seed-pareto-exitoso.png
- 12-conteos-logistica-pareto.png
- 13-conteos-inventario-ventas.png
- 14-get-individual-base-pareto.png
- 15-jmeter-get-smoke-configuracion.png
- 16a-jmeter-get-variables.png
- 16b-jmeter-get-request-configuracion.png
- 17-jmeter-get-smoke-results-tree.png
- 18-jmeter-get-smoke-aggregate.png
- 19-jmeter-get-smoke-summary.png
- 20a-jmeter-get-smoke-corridas-01-02.png
- 20b-jmeter-get-smoke-corridas-03-04.png
- 21a-jmeter-get-baja-corridas-01-02.png
- 21b-jmeter-get-baja-corridas-03-04.png
- 22a-jmeter-get-media-corridas-01-02.png
- 22b-jmeter-get-media-corridas-03-04.png
- 23a-jmeter-get-normal-corridas-01-02.png
- 23b-jmeter-get-normal-corridas-03-04.png
- 23c-jmeter-get-normal-corridas-05-06.png
- 23d-jmeter-get-normal-corridas-07-08.png
- 23e-jmeter-get-normal-archivos-resultados.png
- 24-python-aiohttp-script-ayuda.png

- 25a-python-validacion-get-api-detenido.png
- 25b-python-validacion-get-exitosa.png
- 26-python-validacion-post-24-items.png
- 27a-python-get-alta-corridas-01-02.png
- 27b-python-get-alta-corridas-03-04.png
- 28-python-get-muy-alta-corridas-01-04.png
- 29-python-get-estres-corridas-01-04.png

Anexo B. Trazabilidad de archivos

Elemento	Ruta
Bitácora	pruebas-carga/bitacora.md
Plan GET JMeter	pruebas-carga/jmeter/get-productos-disponibles.jmx
Ejecutor Python	pruebas-carga/python/load_test.py
Prompt IA	pruebas-carga/python/prompt-utilizado.md
Resultados	pruebas-carga/resultados/
Capturas	pruebas-carga/capturas-de-pantalla/

Fin del informe.